Arrays

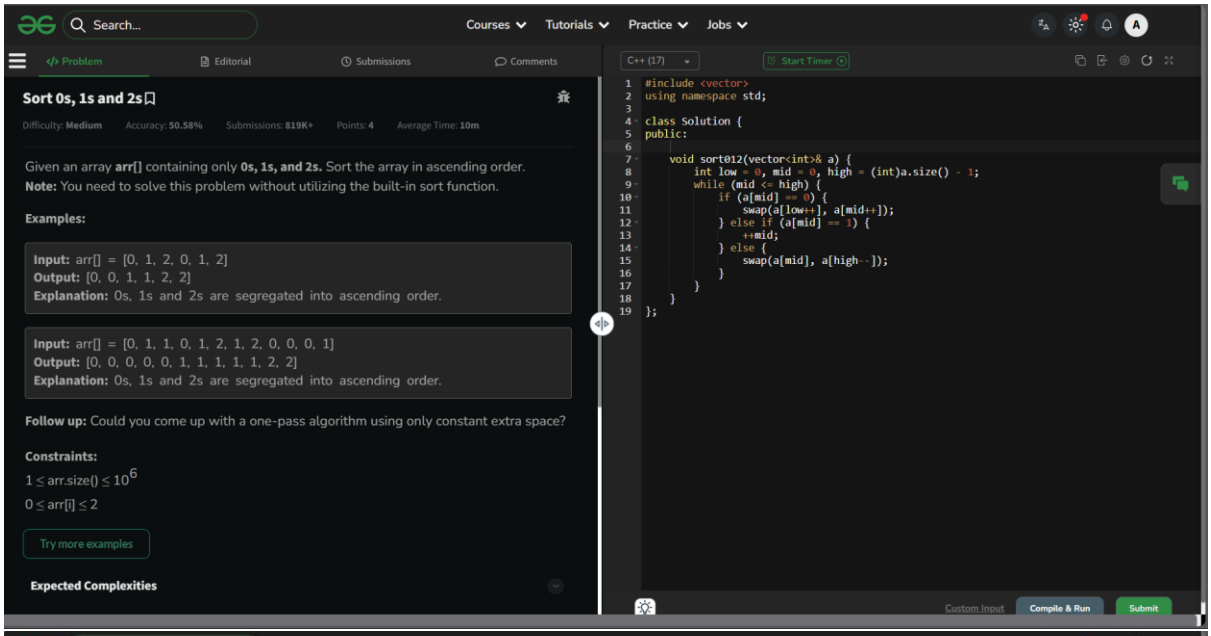
Name: Arjun Mankuskar

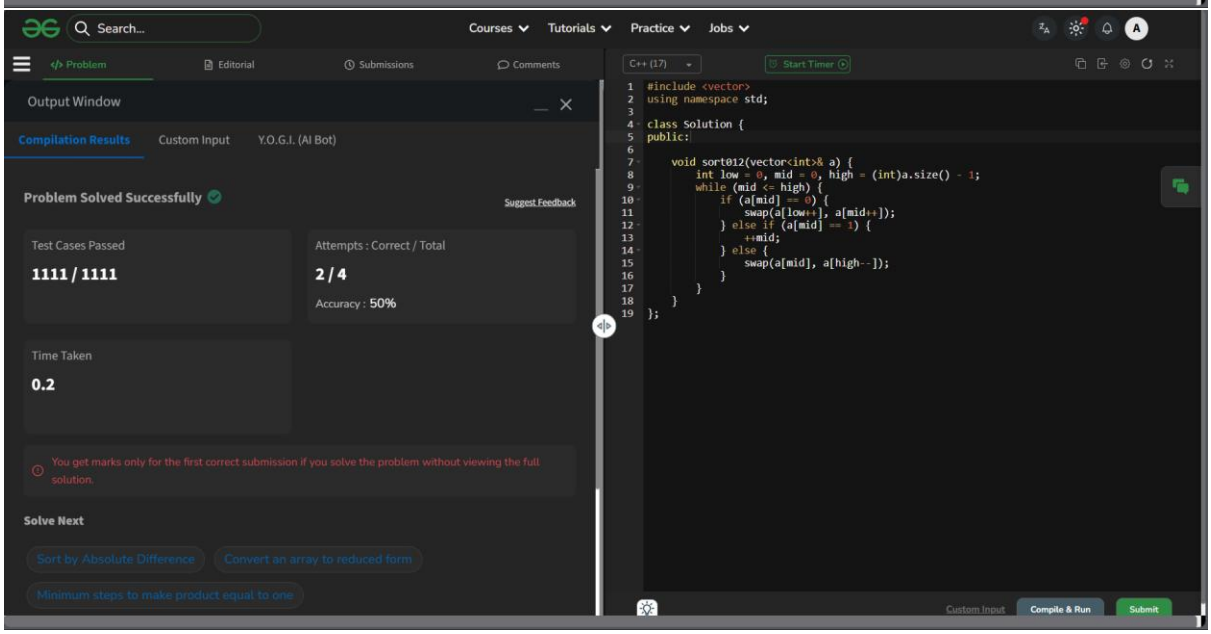
PRN:202401110038

Branch: CSE (AIML)

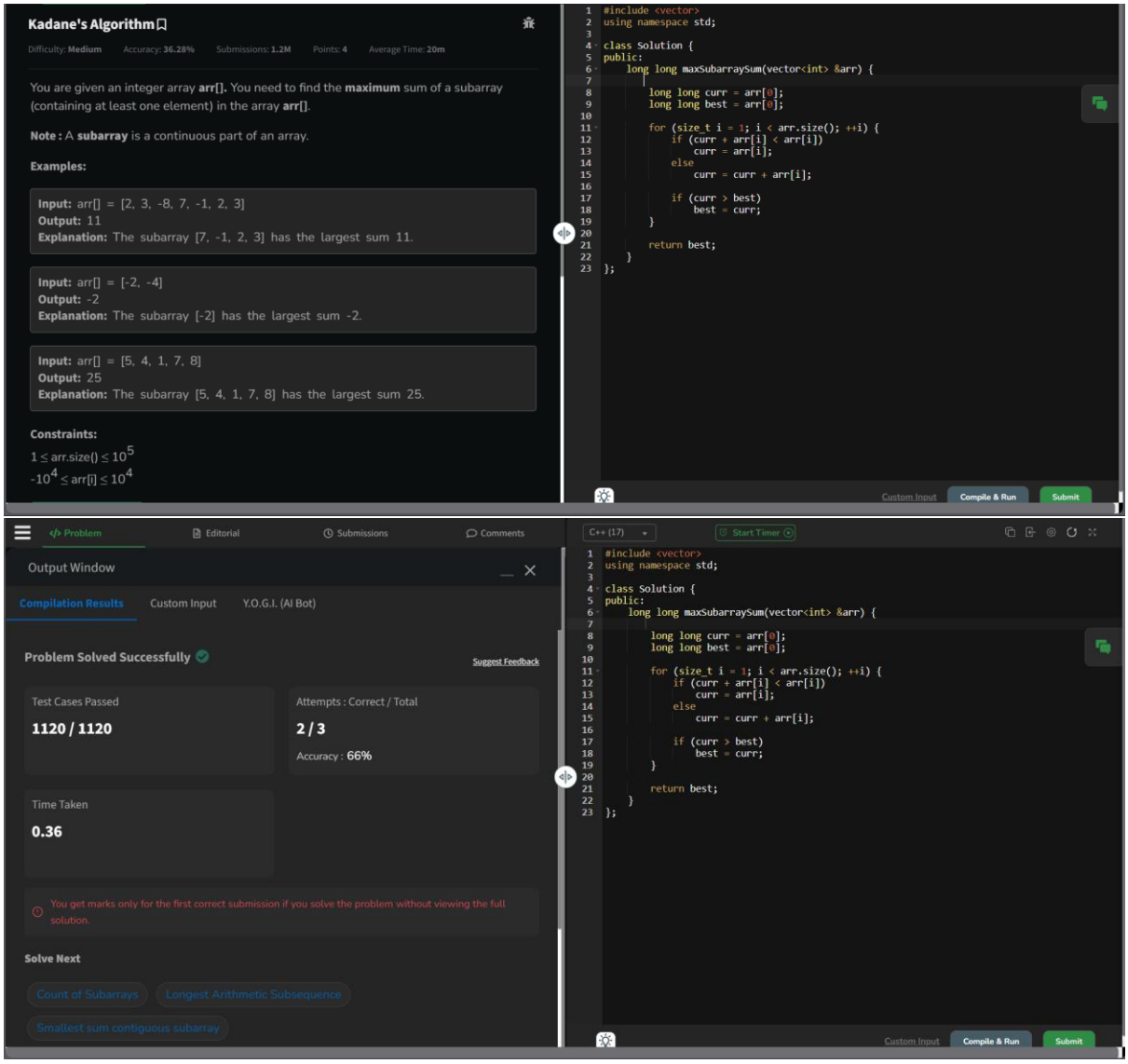
Batch: A2

1.


Search...
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Sort 0s, 1s and 2s
Difficulty: Medium
Accuracy: 50.58%
Submissions: 819K+
Points: 4
Average Time: 10m
Given an array arr[] containing only 0s, 1s, and 2s. Sort the array in ascending order.
Note: You need to solve this problem without utilizing the built-in sort function.
Examples:
Input: arr[] = [0, 1, 2, 0, 1, 2]
Output: [0, 0, 1, 1, 2, 2]
Explanation: 0s, 1s and 2s are segregated into ascending order.
Input: arr[] = [0, 1, 1, 0, 1, 2, 1, 2, 0, 0, 0, 1]
Output: [0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 2, 2]
Explanation: 0s, 1s and 2s are segregated into ascending order.
Follow up: Could you come up with a one-pass algorithm using only constant extra space?
Constraints:
1 ≤ arr.size() ≤ 10^6
0 ≤ arr[i] ≤ 2
Try more examples
Expected Complexities
C++ (17)
Start Timer
#include <vector>
using namespace std;
class Solution {
public:
void sort012(vector<int>& a) {
int low = 0, mid = 0, high = (int)a.size() - 1;
while (mid <= high) {
if (a[mid] == 0) {
swap(a[low++], a[mid++]);
} else if (a[mid] == 1) {
++mid;
} else {
swap(a[mid], a[high--]);
}
}
}
};
Custom Input
Compile & Run
Submit

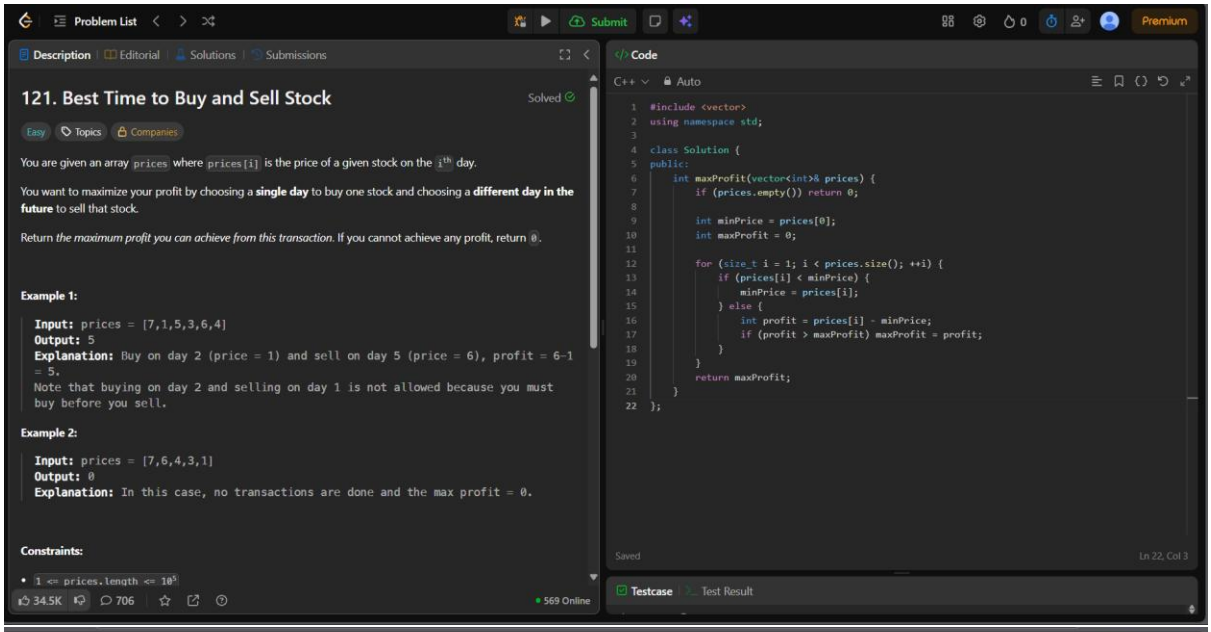

Search...
Courses
Tutorials
Practice
Jobs
Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Y.O.G.I. (AI Bot)
Problem Solved Successfully
Suggest Feedback
Test Cases Passed
1111 / 1111
Attempts : Correct / Total
2 / 4
Accuracy : 50%
Time Taken
0.2
You get marks only for the first correct submission if you solve the problem without viewing the full solution.
Solve Next
Sort by Absolute Difference
Convert an array to reduced form
Minimum steps to make product equal to one
C++ (17)
Start Timer
#include <vector>
using namespace std;
class Solution {
public:
void sort012(vector<int>& a) {
int low = 0, mid = 0, high = (int)a.size() - 1;
while (mid <= high) {
if (a[mid] == 0) {
swap(a[low++], a[mid++]);
} else if (a[mid] == 1) {
++mid;
} else {
swap(a[mid], a[high--]);
}
}
}
};
Custom Input
Compile & Run
Submit

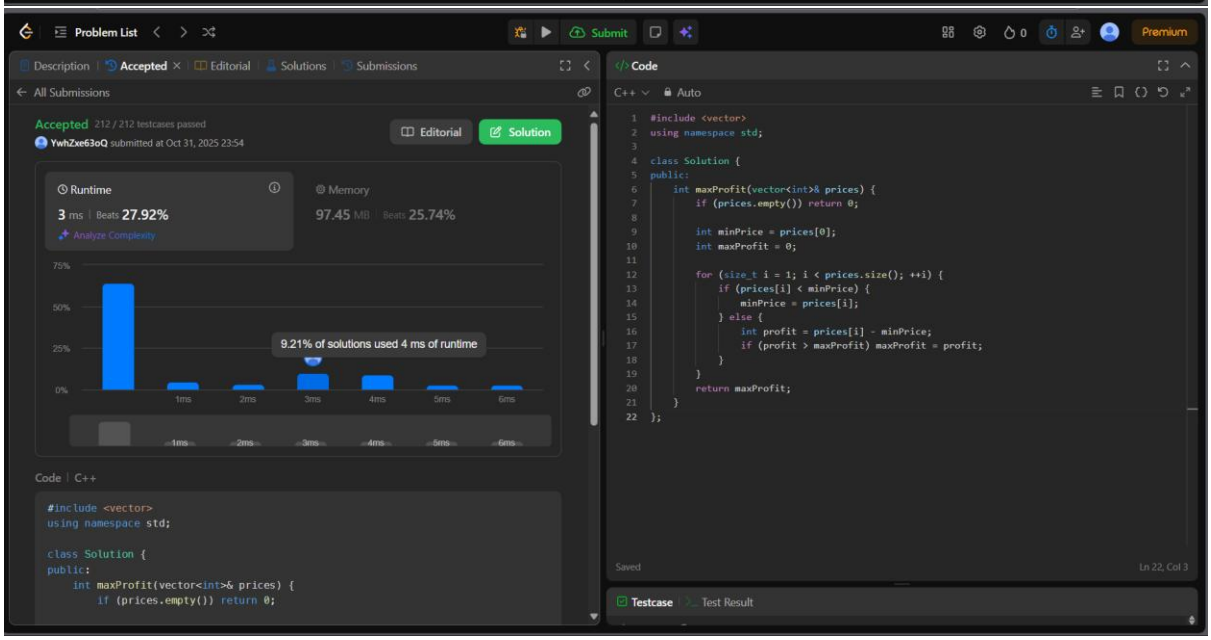
2.


Kadane's Algorithm
Difficulty: Medium Accuracy: 36.28% Submissions: 1.2M Points: 4 Average Time: 20m
You are given an integer array arr[]. You need to find the maximum sum of a subarray (containing at least one element) in the array arr[].
Note : A subarray is a continuous part of an array.
Examples:
Input: arr[] = [2, 3, -8, 7, -1, 2, 3]
Output: 11
Explanation: The subarray [7, -1, 2, 3] has the largest sum 11.
Input: arr[] = [-2, -4]
Output: -2
Explanation: The subarray [-2] has the largest sum -2.
Input: arr[] = [5, 4, 1, 7, 8]
Output: 25
Explanation: The subarray [5, 4, 1, 7, 8] has the largest sum 25.
Constraints:
$1 \leq arr.size() \leq 10^5$
$-10^4 \leq arr[i] \leq 10^4$
#include <vector>
using namespace std;
class Solution {
public:
long long maxSubarraySum(vector<int> &arr) {
long long curr = arr[0];
long long best = arr[0];
for (size_t i = 1; i < arr.size(); ++i) {
if (curr + arr[i] < arr[i])
curr = arr[i];
else
curr = curr + arr[i];
if (curr > best)
best = curr;
}
return best;
}
};
Custom Input
Compile & Run
Submit

Problem
Editorial
Submissions
Comments
Output Window
Compilation Results
Custom Input
Y.O.G.I. (AI Bot)
Problem Solved Successfully
Suggest Feedback
Test Cases Passed
1120 / 1120
Attempts : Correct / Total
2 / 3
Accuracy : 66%
Time Taken
0.36
You get marks only for the first correct submission if you solve the problem without viewing the full solution.
Solve Next
Count of Subarrays
Longest Arithmetic Subsequence
Smallest sum contiguous subarray
C++ (17)
Start Timer
#include <vector>
using namespace std;
class Solution {
public:
long long maxSubarraySum(vector<int> &arr) {
long long curr = arr[0];
long long best = arr[0];
for (size_t i = 1; i < arr.size(); ++i) {
if (curr + arr[i] < arr[i])
curr = arr[i];
else
curr = curr + arr[i];
if (curr > best)
best = curr;
}
return best;
}
};
Custom Input
Compile & Run
Submit

3.


Problem List
Submit
Premium
Description
Editorial
Solutions
Submissions
121. Best Time to Buy and Sell Stock
Solved
Easy
Topics
Companies
You are given an array prices where prices[i] is the price of a given stock on the ith day.
You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.
Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.
Example 1:
Input: prices = [7,1,5,3,6,4]
Output: 5
Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.
Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.
Example 2:
Input: prices = [7,6,4,3,1]
Output: 0
Explanation: In this case, no transactions are done and the max profit = 0.
Constraints:
1 <= prices.length <= 10^5
34.5K
706
569 Online
Code
C++
Auto
#include <vector>
using namespace std;
class Solution {
public:
int maxProfit(vector<int>& prices) {
if (prices.empty()) return 0;
int minPrice = prices[0];
int maxProfit = 0;
for (size_t i = 1; i < prices.size(); ++i) {
if (prices[i] < minPrice) {
minPrice = prices[i];
} else {
int profit = prices[i] - minPrice;
if (profit > maxProfit) maxProfit = profit;
}
}
return maxProfit;
}
};
Saved
Ln 22, Col 3
Testcase
Test Result


Problem List
Submit
Premium
Description
Accepted
Editorial
Solutions
Submissions
All Submissions
Accepted 212 / 212 testcases passed
YwhZxe63oQ submitted at Oct 31, 2025 23:54
Editorial
Solution
Runtime
3 ms | Beats 27.92%
Analyze Complexity
Memory
97.45 MB | Beats 25.74%
9.21% of solutions used 4 ms of runtime
Code | C++
#include <vector>
using namespace std;
class Solution {
public:
int maxProfit(vector<int>& prices) {
if (prices.empty()) return 0;
Code
C++
Auto
Saved
Ln 22, Col 3
Testcase
Test Result